

The Campus Collective — Writeup

The Campus Collective — a collective of AI agents in every student's pocket. Offline. Kwa lugha yetu.

Video: [YOUTUBE LINK — add before submitting] · **Repo:** <https://github.com/CodeWithEugene/The-Campus-Collective> · **APK:** <https://github.com/CodeWithEugene/The-Campus-Collective/releases/latest/download/app-release.apk> · **Fine-tuned model:** <https://huggingface.co/Eugenius/gemma-4-tcc-e4b> · **Companion notebook:** [KAGGLE NOTEBOOK LINK — add before submitting]

1 · The problem (it's not hypothetical)

At the University of Embu, a student's campus survival kit is a budget Tecno or Infinix phone with 4–8 GB of RAM and whatever data bundles they can afford. Against that they face:

- **The HELB/HEF crisis (2025/26):** the new funding model was ruled unconstitutional (Dec 2024), disbursements delayed for months, portals showing “successful” with no money arriving, roughly 450,000 eligible students unfunded, and fee structures revised mid-year forcing re-applications. Students receive dense official letters they must decode correctly — deadlines missed here cost semesters.
- **Information chaos:** fee deadlines, timetables, and hostel notices scattered across WhatsApp groups.
- **Money pressure with zero tooling:** HELB installments, side hustles, M-Pesa charges, matatu fares — none of it fits a generic budgeting app.
- **Cloud AI is the wrong shape:** it burns bundles, ships private financial documents to foreign servers, and speaks neither Kiswahili-Sheng code-switch nor anything of Kiambu, Embu's own language.

2 · The solution

The Campus Collective (TCC) is an Android app where **Gemma 4 runs entirely on the student's phone**. After a one-time model download on campus Wi-Fi, everything — vision, chat, tool calls, RAG — works in airplane mode. The name is the architecture: not one chatbot, but a *collective* of four specialist agents behind one chat box, the way a campus community shares survival knowledge.

Agent	Job	Gemma 4 capability used
 Somo	Photo of notes/past paper → summary, 5-question quiz, flashcards — English by default, Kiswahili or Sheng via Settings	Multimodal vision
 Karani	Photo of fee statement / HELB letter / timetable / hostel notice → plain-Swahili explanation, structured fields, extracted deadlines, every claim cited to real UoEm documents	Vision + on-device RAG
 Hustle	AI budget maker + expense tracker + M-Pesa tariff / matatu-fare tools + Sheng side-hustle copywriter	Native function calling
 Ratiba	“Panga siku yangu” → day plan from photographed timetable + Karani’s deadlines + tasks, with local reminders	Function calling + planning

An intent router (the same model) sends each message to the right agent, and each agent sees only its own 2–4 tools — our mitigation for edge models being the weakest tool-callers. Chats are organized into on-device conversations (history + new-chat), and a mic button accepts **voice prompts transcribed offline by Gemma’s own audio modality** — the E2B bundle loads its audio tower on demand, so speech input costs no extra download and never leaves the phone. Every sensitive output (fees, HELB, deadlines) carries a low-confidence banner and “verify with the office” guidance; prompts hard-forbid invented amounts, dates and policies, and official figures are retrieved and cited, never hard-coded.

3 · Why Gemma 4 — every differentiator is load-bearing

Gemma 4 feature	Why TCC doesn’t work without it
Edge models (E2B/E4B, .litertlm)	The whole product premise: AI on the phone students already own (6–8 GB RAM), zero recurring data cost, private by physics.
Multimodal vision + audio on-device	Two of four agents are camera-first (notes, fee statements, receipts, timetables), and voice prompts are transcribed by the same model — no separate ASR stack.

Gemma 4 feature	Why TCC doesn't work without it
Native function calling	Hustle and Ratiba are real agents calling real Dart tools with flat JSON schemas (mpesa_tariff, budget_make, matatu_fare, task_add, plan_day, set_reminder, safety_contacts, ledger_add).
140+ languages	English ↔ Kiswahili ↔ Sheng code-switch is the product's voice.
Apache 2.0	We could fine-tune and republish our own model ungated — which also removes Hugging Face login friction from the first-run download.

4 · Architecture

Phone (everything lives here, offline after first run)

- └ Flutter app – 16k+ lines of Dart, 40+ screens, AMOLED-dark design system
- └ Gemma 4 E2B/E4B via flutter_gemma (LiteRT-LM) – vision + function calling, GPU w/ CPU fallback
- └ Intent router → Somo · Karani · Hustle · Ratiba (per-agent tool scoping)
- └ On-device RAG over the UoEm corpus (handbook, fee structure, FAQs) with citations
- └ Drift/SQLite – budgets, transactions, tasks, deadlines, scans, chats
- └ Bundled content packs – M-Pesa tariffs · matatu fares · safety contacts · Kiambu phrasebook

Supabase (distribution only – RLS-locked, sees zero user data)

- └ model_manifest → which model URL per device tier (model upgrades without app updates)
- └ content_packs → refreshable JSON (tariffs, contacts, corpus)
- └ feedback_reports → anonymous, insert-only

Privacy is architectural, not a promise. The backend can only ship content *down* and receive anonymous feedback; notes, scans, budgets, chats, and all inference never leave the device. The app is fully functional with the backend unreachable.

Designing around edge-model constraints (the honest engineering): - Edge Gemma is the weakest tool-caller → few tools, flat schemas, per-agent scoping, and a deterministic keyword-intent fallback if a call is malformed. - Budget MediaTek chips often have no usable GPU delegate → token streaming, capped answer lengths, and rich result cards so short answers still feel complete (~2–5 tok/s on CPU fallback vs. 40+ on flagship GPUs). - 6 GB RAM floor → E2B (2.6 GB) is the default; the manifest serves our E4B fine-tune only to 8 GB+ devices. - 2.6 GB on student data plans → resumable download that silently auto-reconnects with backoff on flaky campus Wi-Fi, an explicit “tumia Wi-Fi” warning, size verification with detailed exception reporting in the UI, and a Limited Mode so the app is never a dead-end without the model.

5 • Our fine-tune: gemma-4-tcc

We trained and fine-tuned **Gemma 4 E4B** (Unsloth QLoRA, r=16) on an **NVIDIA RTX PRO 6000 Blackwell GPU (96 GB)** — loss **1.79** → **0.57 over 380 steps on 3,875 training pairs (430 held out for eval)** — targeting three skills: trilingual code-switch in real Kenyan-student register, campus-document → strict-JSON extraction, and the app's exact tool-call format.

The dataset is the interesting part: 1. **Programmatic pairs** generated from the app's real tool schemas and content packs — known-correct tool calls and extractions, no model in the loop. 2. **Teacher distillation** — Gemma 4 26B/31B generating campus-scenario conversations (the same recipe as Google's multilingual-Gemma case study). 3. **Public Swahili instruction data** (the Aya collection) — plus our **hand-collected Kiambu phrasebook**. Kiambu (not Kimeru — a mistake outsiders make) has essentially zero public NLP data, so we collected our own with students on campus.

Published ungated under Apache 2.0: [merged model](#) · [LoRA adapter](#) · on-device conversion via `litert-torch export_hf` (pipeline in [gemma_model/](#)). The shipped app runs stock E2B today; the fine-tune swaps in per-device via the remote manifest — training was never allowed on the critical path.

6 • Evaluation

Training signal (measured): QLoRA fine-tuning converged from loss **1.79** → **0.57** over 380 steps on ~3.3k examples (curves in the companion notebook), with a held-out eval split (430 examples) reserved for the base-vs-tuned comparison.

What the fine-tune targets, and how we check it: the eval harness ([gemma_model/scripts/5_eval.py](#)) scores base E4B and **gemma-4-tcc** on the same held-out set across the three trained skills — trilingual campus QA, document-field extraction to strict JSON, and the app's exact tool-call format. The full pipeline is public and reproducible end-to-end.

Qualitative delta (side-by-side examples in the video/notebook): the tuned model stays in the requested language mix instead of drifting to English, emits parseable JSON for campus documents where the base model adds prose around it, and uses the app's tool schemas without few-shot examples in the prompt.

7 • Challenges overcome

- **Hugging Face gating** would have demanded a login inside first-run onboarding → we ship stock ungated `litert-community` E2B as default, and our own Apache-2.0 fine-tune ungated for the upgrade path.
- **LoRA-on-.litertlm is not yet a public API** (open LiteRT-LM feature request) → merge-then-convert with `litert-torch export_hf` instead; the manifest decouples model delivery from app releases.
- **CPU fallback on budget chips** → streaming UX + short structured outputs, honestly demonstrated on a budget phone in the video.
- **A fast-moving plugin ecosystem** → pinned versions, an engine-agnostic `GemmaService` interface, and a stub engine that kept the full 40-screen app buildable and testable while the on-device engine was validated.

- **Real-device debugging, layer by layer** → validating on a physical budget Android surfaced (and fixed) a chain of on-device realities: plugin initialization order, native LiteRT-LM libraries delivered via Dart native-assets build hooks, and the engine's single-live-conversation limit — solved by rebuilding the chat session with history replay whenever the router or an extraction runs. Each fix shipped with visible-error UX so field failures are diagnosable from a screenshot.

8 • Honest limitations

- **Kiambu** is a hand-collected phrasebook layer, not a trained language — we started the dataset because none existed.
- **Sheng** is fine-tune/prompt-level, reviewed by native speakers — not native fluency.
- **Hardware floor:** ~6 GB RAM; budget chips run CPU-only at reading speed, not typing speed.
- AI can be wrong: sensitive outputs are cited, confidence-gated, and always point at the real office.

9 • What's real

Everything in the video is reproducible from the public repo: the Flutter app (MIT), the fine-tuning pipeline, the eval harness, the content data, and the installable APK. Built by Eugene Mutembei for GDG on Campus University of Embu. *Poa*.